Jaypee Institute of Information Technology, Noida

2026-27

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION TECHNOLOGY

Project Title: SecureShare - An Encrypted File Sharing System



PROJECT SYNOPSIS

Submitted by: Enrollment No.: 992501230050, Name: Labhansh Vashisht

Enrollment No.: 992501230060, Name: Mohd. Nauman Ali

Course Name: Python Lab

Course Code: 26B15CS217

Program: Btech CSE (Cyber Security)

Second Year (3rd Semester)

1. Problem Statement and Objectives

Problem Statement:

In the modern digital age, secure file sharing remains a critical challenge for individuals and organizations. Most conventional file-sharing platforms store files on centralized cloud servers, exposing sensitive data to potential breaches, unauthorized access, and privacy violations. Users often lack control over their data and cannot verify whether their files remain confidential during transmission and storage.

Objectives:

Develop a secure, end-to-end encrypted file sharing application that enables users to upload, store, and download files with complete confidentiality and integrity assurance.

Implement robust cryptographic mechanisms (AES encryption, RSA key exchange, and SHA-256 hashing) to protect sensitive data throughout the entire lifecycle of file transmission and storage.

Create a user-friendly interface that abstracts complex cryptographic operations, allowing non-technical users to securely share files without understanding the underlying security mechanisms.

Demonstrate practical application of Python programming concepts including networking, file handling, object-oriented design, and cryptographic libraries.

2. Proposed Solution and Architecture

SecureShare is a client-server Python application that implements end-to-end encryption for secure file sharing. The system employs a hybrid encryption model combining symmetric (AES) and asymmetric (RSA) cryptography to ensure confidentiality, authenticity, and integrity of transmitted files.

System Architecture:

The application follows a three-tier architecture consisting of client applications, a central server, and encrypted storage. Users authenticate via secure credentials, select files for sharing, and the system automatically encrypts files using AES-256 before transmission. The AES encryption key is itself encrypted using the recipient's RSA public key, ensuring only the intended recipient can decrypt and access the shared file. File integrity is verified using SHA-256 cryptographic hashing.

Key Features:

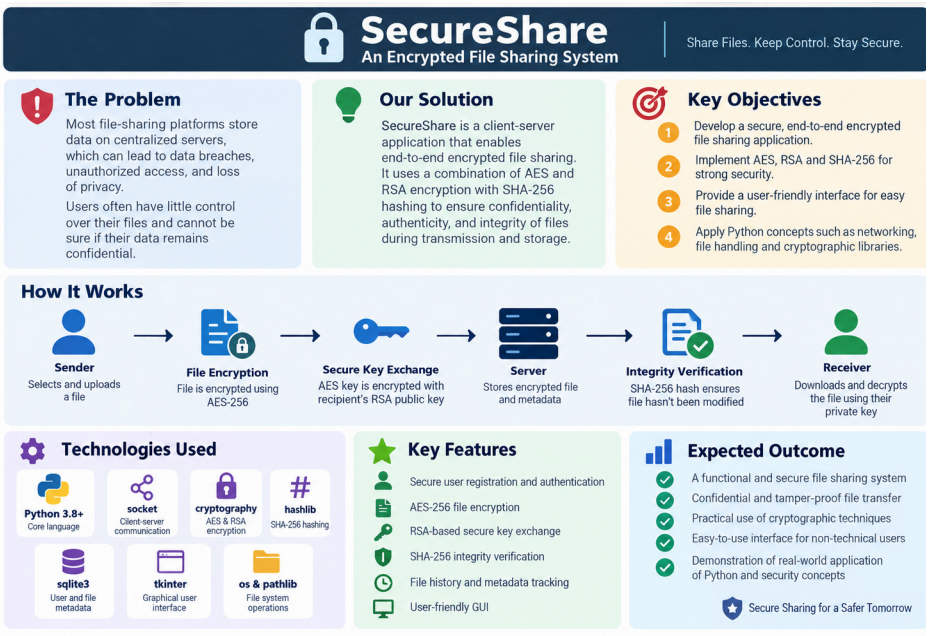
User Management: Secure user registration and authentication with password hashing using industry-standard algorithms.

File Encryption: AES-256 symmetric encryption for actual file content, providing high-speed encryption suitable for large files.

Secure Key Exchange: RSA public-key cryptography for secure transmission of AES keys between users without direct key sharing.

Integrity Verification: SHA-256 hashing to detect any unauthorized modifications to transmitted or stored files.

File History and Metadata: SQLite database to track uploaded, downloaded, and shared files with timestamps and recipient information.



3. Proposed Technologies and Tools

Programming Language: Python 3.8+

Our project leverages Python's extensive standard library and third-party cryptographic packages to implement secure file sharing with minimal dependencies. Python's readability and powerful abstraction capabilities make it ideal for demonstrating complex cryptographic concepts.

Core Libraries and Frameworks:

socket: Low-level client-server networking for secure TCP communication between clients and the central server.

cryptography (pyca/cryptography): Industry-standard cryptographic library implementing AES encryption, RSA key generation and encryption, and related secure operations.

hashlib: Built-in module for SHA-256 and other cryptographic hash functions to verify file integrity.

sqlite3: Lightweight relational database for storing user credentials, file metadata, and transaction history.

tkinter: Built-in GUI framework to create an intuitive graphical interface for both file upload/download and user authentication.

os & pathlib: Standard libraries for secure file system operations and path handling.

4. Methodology and Implementation Plan

The project will be developed in eight iterative phases, each building upon previous components to create a fully functional, secure file sharing system:

Phase 1: Basic Client-Server Communication

Establish socket-based TCP communication between client and server, implementing message protocols for file transfer requests and responses.

Phase 2: User Authentication System

Implement user registration and login functionality with password hashing using bcrypt or PBKDF2, backed by SQLite database for user credential storage.

Phase 3: AES File Encryption
Implement symmetric AES-256-CBC encryption for file content, including initialization vector (IV) generation and secure key management practices.

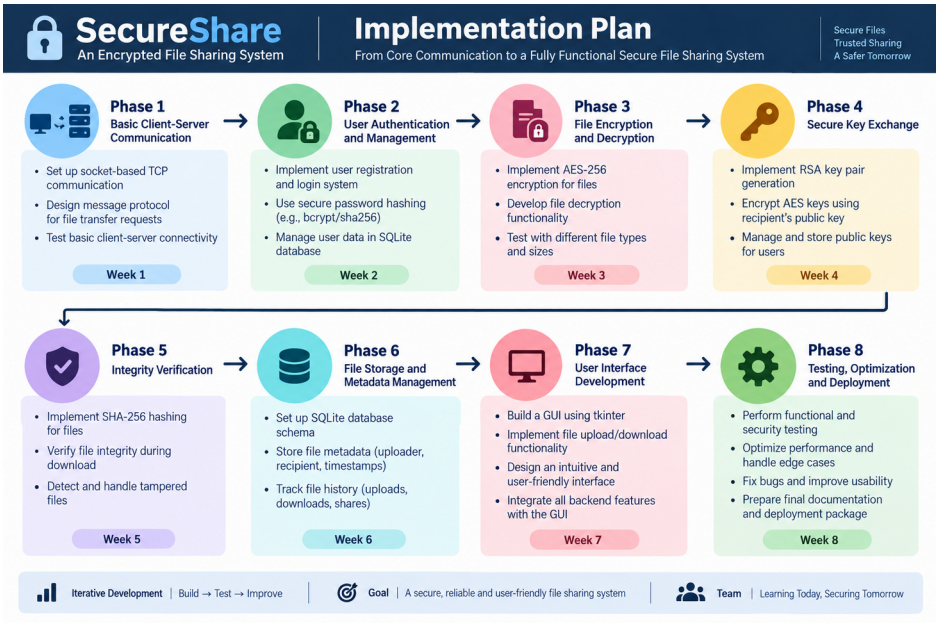
Phase 4: RSA Key Pair Generation and Secure Key Exchange
Generate 2048-bit RSA key pairs for each user, store public keys on the server, and implement secure encryption of AES keys using recipient's public key.

Phase 5: SHA-256 Integrity Verification
Compute SHA-256 hashes of original files, transmit alongside encrypted files, and verify integrity upon decryption to detect tampering.

Phase 6: Graphical User Interface Development
Develop tkinter-based GUI with intuitive windows for authentication, file upload/download selection, recipient selection, and file history viewing.

Phase 7: Database Integration and File History
Implement SQLite database schema to track sent/received files, timestamps, recipient information, file sizes, and encryption metadata.

Phase 8: Security Demonstration and Documentation
Demonstrate security features by intercepting encrypted file transfers, showing unreadable ciphertext, attempting tampering, and showing hash verification detection. Comprehensive documentation of system architecture, security protocols, and user manual.



5. Expected Outcomes and Deliverables

Upon successful completion, the project will deliver:

A fully functional, end-to-end encrypted file sharing application demonstrating practical implementation of Python programming concepts and cryptographic principles.

Complete source code with well-documented modules explaining cryptographic implementations, networking protocols, and database operations.

User documentation and system architecture diagrams illustrating data flow, encryption pipelines, and security mechanisms.

Live demonstration showcasing secure file transfer, encryption/decryption, integrity verification, and security attacks/detection.

6. Sustainable Development Goal (SDG) Alignment

Primary SDG: SDG 9 – Industry, Innovation and Infrastructure
SecureShare directly supports SDG 9 by developing innovative digital infrastructure for secure information exchange. The project demonstrates how thoughtful application of cryptographic technology can build reliable, secure digital systems that enable trustworthy communication between individuals and organizations. This aligns with SDG 9's focus on building resilient infrastructure and fostering innovation through technology.

Secondary SDG: SDG 16 – Peace, Justice and Strong Institutions
Secure and transparent communication systems are foundational to trustworthy institutions. By enabling confidential and tamper-proof file exchange, SecureShare contributes to institutional accountability and the protection of sensitive information. Organizations can rely on secure digital infrastructure to conduct transparent operations while protecting citizen privacy and organizational confidentiality.

7. Project Timeline and Milestones

Week 1 (Sept 7-8): Design and Architecture Planning

Finalize system architecture, create detailed design diagrams, and plan database schema.

Week 2 (Sept 9-11): Core Development (Phases 1-3)

- Implement client-server communication, user authentication, and AES encryption modules.

Week 3 (Sept 12 onwards): Advanced Features and Integration (Phases 4-6)

Develop RSA key exchange, SHA-256 verification, and graphical user interface.

Week 4: Final Integration and Demonstration (Phases 7-8)

Complete database integration, perform comprehensive testing, prepare security demonstration, and finalize documentation.

8. Conclusion

SecureShare represents a comprehensive application of Python programming concepts to solve a real-world problem in digital security. By implementing industry-standard cryptographic protocols and creating an accessible user interface, this project demonstrates the practical relevance of computer science education to contemporary challenges in data protection and privacy. The project successfully bridges theoretical cryptographic principles with practical implementation, enabling stakeholders to understand and trust secure digital infrastructure. Through this endeavor, we contribute meaningfully to SDG 9 and SDG 16 by advancing technological innovation and institutional trust.